

Набор инструкции за ARM процесор

Дизайнерът на микропроцесора избира инструкциите с голямо внимание. Трябва лесно да изпълняват сложни операции като последователност от прости събития, всяко от които се представя с една инструкция от добре проектирания набор от инструкции.

От появата на микропроцесора в началото на 70-те години се наблюдава тенденция наборите от инструкции да стават все по-сложни. В началото на 80-те години се появява различен подход към набора от инструкции. Създаден бе компютър с намален набор от инструкции /RISC/.

Повечето програми през над 80% от времето изпълняват инструкции за прехвърляне на данни, предаване на управлението или аритметични инструкции. Ако бъде намерен метод за увеличаване на скоростта на тези инструкции, програмата ще се изпълни много по-бързо. Второто наблюдение беше за използването на константни стойности. Около 56% от константите в програмите са в диапазона от ± 15 , а 98% от константите са в диапазона ± 11 . Тъй като обикновено константните стойности са достатъчно малки (само 5 бита), те могат да бъдат включени като част от инструкцията, вместо да се извличат от паметта. Последното наблюдение бе, че броят на параметрите (аргументите), предавани между подпрограмите, обикновено е по-малък от шест. По този начин, ако процесорът има достатъчно регистри, не е необходимо да се използва външна (извън чипа) памет за стек.

Тези три наблюдения формират основата на т.нар. Berkeley RISC архитектура, а впоследствие принципите, въз основа на които е проектиран наборът от инструкции ARM. Наборът от инструкции за ARM процесора може да бъде разделен на шест широки класа инструкции /виж Табл. 4.2/.

Прехвърляне на данни

MOV Move

Аритметични

ADD Add

ADC Add with Carry

SUB Subtract

SBC Subtract with Carry

RSB Reverse Subtract

RSC Reverse Subtract with Carry

MUL Multiply

MLA Multiply Accumulate

Управление на хода на програмата

CMP Compare

CMN Compare Negative

TEQ Test Equivalence

TST Test

BAL Branch Always

B<cc> Conditional Branch

BLAL Branch and Link Always

BL<cc> Conditional Branch and Link

Достъп до паметта

LDR Load Register

STR Store Register

LDRB Load Register Byte

STRB Store Register Byte

LDM Load Multiple

STM Store Multiple

Логически и за манипулация на битове

AND Bitwise AND

BIC Bit Clear

ORR Bitwise (inclusive) OR

EOR Bitwise Exclusive OR

MVN Move Negative

Управление на системата

MRS Move to Register from Status

MSR Move to Status from Register

SWP Swap

SWPB Swap Byte

SWI Software Interrupt

Табл 4.2: Мнемоники на инструкциите

4.1 Инструкции за прехвърляне на данни

Има само една инструкция, която попада в тази категория: MOV (Move) инструкция. Това обаче е може би най-важната инструкция, тъй като над 40% от програмата ще използва тези инструкции. Нека разгледаме тази инструкция по-подробно, тъй като тя има две общи полета с повечето други инструкции, а именно поле на зададените флагове и поле за условно изпълнение.

Синтаксисът на тази инструкция е:

MOV <cc> <S> **Rd**, <op1>

където полетата означават:

MOV това е мнемоника за инструкцията, която показва коя операция ще извършим. Оригиналната стойност не се унищожава, а се копира.

<cc> обозначава условно изпълнение. Инструкцията се изпълнява само при определеното условие. Това е или двубуквен код на условие, или липсва изобщо. В този случай се приема, че инструкцията трябва да се изпълни винаги.

<S> показва, че операцията трябва да установи кода на условието. Той се използва при условното изпълнение.

Rd обозначава регистъра на получателя. Това винаги е регистър и може да бъде всеки регистър, наличен в текущия режим на процесора.

<op1> показва източника на данните. Инструкцията MOV е необичайна с това, че има само един източник. Повечето инструкции изискват два операнда, следователно те изискват два източника. Първият обикновено е регистър на източника (Rs).

Изходният регистър (ако има такъв) се копира в регистър A на ALU. Източникът <op1> се копира в регистър B на ALU, което означава, че трябва да премине през Barrel Shifter. Това позволява на програмиста да подготви аритметично или логическо изместване на данните преди това. В резултат няма изискване за специални инструкции за смяна, намиращи се в други (CISC) процесори.

Вземайки това предвид, схематичното описание на обща MOV инструкция, би било:

<cc>: $ALU(B) \leftarrow <op1>$

<cc>: $Rd \leftarrow ALU$

<cc><S>: $CSPR \leftarrow ALU(flags)$

където <cc> показва условно изпълнение, а <S> указва установяване на флаговете след изпълнението на инструкцията.

4.1.1 Варианти на флаговете

Повечето от инструкциите за обработка на данни, като групите аритметични и логически инструкции, могат да установяват флаговете на кода на условието. Това се обозначава с <S> в синтаксиса на инструкцията и се активира чрез добавяне на буквата S в края на мнемониката.

Общо	Със знак	Без знак
CS Carry Set	GT Greater Than	HI Higher Than
CC Carry Clear	GE Greater Than or Equal	HS Higher or Same (aka CS)
EQ Equal (Zero Set)	LT Less Than	LO Lower Than
NE Not Equal (Zero Clear)	LE Less Than or Equal	LS Lower or Same (aka CC)
VS Overflow Set	MI Minus (Negative)	
VC Overflow Clear	PL Plus (Positive)	

Табл. 4.3: Мнемоники <cc> за кода на условието

Инструкциите за управление на хода на програмата винаги влияят върху състоянието на процесора, така че не ползват вариант при изпълнението си. Инструкциите за достъп до паметта не предават данните през ALU и така не могат да модифицират флаговете. Това е означава, че трябва да се добави допълнителна инструкция, за да се провери дали стойността, която е прочетена от паметта, е нула или не. Инструкцията MOVS ще предаде стойността през ALU, за да генерира кодовете на условието.

По-специално може да се зададат само флагове за нула и знак. Този вариант може да се комбинира с функцията за контрол на хода на изпълнение и условно изпълнение, за по-бърз и ефективен код.

Можем да посочим действието, зададено при установени флагове, като използваме полето <S> в схематичното описание на инструкцията:

<S>: CSPR $\leftarrow$ ALU(flags)

4.1.2 Условно изпълнение: <cc>

По-голямата част от инструкциите включват поле код на условие (<cc>), което позволява на инструкцията да бъде изпълнена условно в зависимост от текущото състояние на процесора (флагове на кода на условието). Ако флаговете показват, че условието е изпълнено, инструкцията се изпълнява, в противен случай инструкцията се игнорира и процесорът просто преминава към следващата инструкция. Списък на двубуквените кодове на условия е показан в таблица 4.3 по-горе. За да посочим, че дадена инструкция е условна, поставяме мнемоника с две букви за условието след мнемониката на инструкцията, но преди <S> полето, ако присъства. Ако не се използва мнемоника за код на условие, инструкцията винаги ще се изпълнява. Има и мнемоника от две букви, която да обозначава винаги (AL). Условиата по-голямо и по-малко са за използване при числа със знак, докато условия по-висок и по-нисък са за използване при числа без знак. Тези кодове на условия имат смисъл само след сравнение.

Например инструкцията

MOVCC R0, R1

ще копира стойността в регистъра R1 в регистъра R0, само когато флагът за пренос е нулиран (код на условието CC). R0 ще остане непроменен, ако флаг C е установен. Условното изпълнение може да се комбинира с инструкцията за вариант <S>.

Например следният фрагмент от кода:

MOVS R0, R1

MOVEQS R0, R2

MOVEQ R0, R3 ще прави следното:

1. Първата инструкция винаги ще копира R1 в R0, но също така ще установи съответно флаговете N и Z.
2. Втората инструкция се изпълнява само ако е установен Z флага, т.е. стойността на R1 е нула. Ако стойността на R1 не е нула, инструкцията се пропуска. Ако се изпълни втората инструкция, тя ще копира стойността на R2 в R0 и също така ще установи флаговете N и Z според стойността на R2.
3. Третата инструкция се изпълнява само ако R1 и R2 са нула.

4.2 Аритметични инструкции

Всички осем инструкции в тази група формират аритметична операция върху два (и в един случай три) изходни операнда, единият от които трябва да бъде регистър (Rn), а друга стойност (<op1>), произвеждаща резултат, който се поставя в регистъра на получателя (Rd). Те могат също така по избор да актуализират флаговете на кода на условието въз основа на резултата.

4.2.1 Събиране

Има две инструкции за събиране:

ADD (Add) и ADC (Add with Carry). И двете инструкции добавят две 32-битови числа, но могат да бъдат комбинирани, за да добавят 64-битови или по-големи числа. Нека регистрите R0, R1 и R2, R3 съдържат две 64-битови числа, с младшата дума в регистрите R0 и R2 съответно. Можем да съберем тези 64-битови числа, за да получим като сума 64-битово число в R4, R5:

ADDS R4, R0, R2 ; Събира младшите думи

ADC R5, R1, R3 ; Събира старшите думи

ADDS събира двете по-младши думи, но също така установява флага за пренасяне, ако има пренос, и го изчиства, ако няма пренос. ADC събира горните думи, но добавя и преноса, получен при предишното събиране.

4.2.2 Изваждане

Има четири инструкции за изваждане.

Инструкциите SUB (Изваждане) и SBC (Изваждане със заем) са еквивалентни на инструкциите ADD и ADC по-горе.

За разлика от събирането, където резултатът от събирането на $4+2$ е същият като този от добавянето на $2+4$, /не зависи от смяна на мястото на операндите/ резултатът от изваждане на $4-2$ (2) не е същият като изваждане на $2-4$ (-2). За да се реализира това изваждане, имаме две допълнителни „обратни“ инструкции: RSB (обратно изваждане) и RSC (обратно изваждане със заем), които изваждат стойността в регистър от друга стойност.

При изваждане флагът за пренос съдържа заема от следващия (32-ри) бит. По този начин инструкциите SUB и SBC могат да бъдат комбинирани по същия начин като инструкциите ADD и ADC, за да се направи 64-битово изваждане. Инструкцията RSB и RSC също могат да бъдат комбинирани по този начин.

4.2.3 Умножение

Има две инструкции за умножение, MUL (умножение) и MLA (умножение с натрупване). Тези инструкции се различават от останалите в тази група, тъй като могат да работят само с регистри. Вторият операнд трябва да бъде регистър, а не стойност <op1>, както се използва от другите инструкции в аритметичната група.

Инструкцията MUL ще използва Booth Multiplier устройството, за да умножи две 32-битови числа, давайки 32-битов резултат. Това създава малък проблем, тъй като умножаването на две 32-битови числа трябва да даде 64-битов резултат. Така че инструкциите за умножение трябва да се използват с повишено внимание. Това обаче означава, че можем да умножим до 16-битови числа със знак и да получим правилен 32-битов резултат със знак.

Инструкцията MLA е същата като инструкцията MUL с изключение на това, че ще добави резултата от умножението към стойността записана в трети регистър. Това може да се използва за изчисляване на текущи суми.

4.2.4 Делене

Няма инструкции за делене.

Делението се извършва значително по-рядко от другите аритметични операции. Възможно е обаче да се извърши деление чрез многократно изваждане на делителя от делимото, докато резултатът стане нула или по-малък от делителя. Броят пъти, когато

делителят е изваден, е известен като частно, а стойността, останала след последното изваждане, е остатъкът. Тоест: делимо = частно * делител + остатък

Нека разгледаме следното уравнение: $10/7 = 1$ остатък 3.

В това уравнение делимото е 10, делителят е 7, резултатът е в две части: частното (1) и остатък (3). Нека имаме числа със знак.

При деление със закръгляне надолу частното се закръгля надолу към отрицателна безкрайност, а остатъкът се използва за коригиране на закръгляването. По този начин, ако разделим -10 на 7, ще имаме частно от -2 и остатък от 4 ($7 * (-2) = -14 + 4 = -10$).

При симетрично деление закръгляването се извършва по обратния начин, към нула.

Така -10, разделено на 7, ще има частно -1 с остатък -3 ($7 * (-1) = -7 + (-3) = -10$).

Подобно на изваждането, деленето не е комутативно.

4.3 Управление на хода на програмата

Инструкциите за управление на хода на програмата попадат в две подгрупи: тези, които сравняват две стойности и задават кодовете на условия и тези, които променят хода на изпълнение (преход).

4.3.1 Сравнение

Има четири инструкции за сравнение, които използват същия формат като аритметичните инструкции. Те изпълняват аритметична или логическа операция върху два изходни операнда, единият от които трябва да бъде регистър, а другият може да бъде всяка стойност <op1>. Инструкциите не записват резултата в регистър, а го използват за актуализиране на флаговете на условието.

Инструкциите CMP (Compare) сравняват две числа. Те ще извадят втория операнд (стойността <op1>) от първия (регистъра), установявайки съответно всички флагове на състоянието:

Zero Установява се, ако двете числа са равни, в противен случай е нула.

Negative – Установява се ако вторият операнд по-голям от първия и се нулира, ако е по-малък.

Carry- Установява се, ако при изваждането има заем

oOverflow – Установява се при препълване.

Това ще позволи да се използва в инструкциите, които следват пълният набор от кодове на условия, за двоични числа, числа със знак или числа без знак.

Инструкцията CMN (Compare Negative) сравнява стойността на регистъра с минуса на втората стойност. По този начин може да се използва за промяна на знака на втората стойност. Това може да предизвика затруднения, ако стойността е нула.

Инструкцията TEQ (Test Equivalence) проверява дали двете стойности са равни. Той ще установи Z флага, така че могат да се използват кодовете на условие EQ (равно) и NE (не е равно). Може да се използва и за проверка дали две стойности имат един и същ знак. Флагът N ще бъде нула, ако двете стойности имат един и същ знак, така че може да се използва условието PL (плюс). Ако двете стойности имат различни знаци, флагът ще бъде нула и може да се използва кодът на MI (Минус).

Последната инструкция в тази подгрупа е TST (Test) инструкция, която се използва за откриване дали един бит от регистъра е установен или не. Флаг Z ще бъде установен, ако битът в регистъра е изчистен и може да се използва кодът на условие EQ (Equal). Ако битът в регистъра е установен, флагът ще бъде нула и може да се използва кодът на условието NE (не е равно).

TST може да се използва за тестване на битове. Всички битове трябва да са нули, за да бъде установен Z флагът. Ако някой от битовете на регистъра е установен, флагът ще бъде изчистен.

4.3.2 Разклонение

Инструкцията B<cc> (Branch) се използва за промяна на реда на изпълнение на инструкциите чрез добавяне на отместване към текущата стойност на програмния брояч. Отместването е число със знак, което позволява преход напред и назад до 32MB. В асемблера поставяме етикет след инструкцията B <cc> (Branch) и асемблерът ще изчисли отместването вместо нас.

Инструкцията Branch може да се използва по два начина, условно и безусловно. Безусловният преход се прави винаги, независимо от стойността на флаговете. Това се записва като BAL (Branch Always).

За условно разклонение един от кодовете на условията, изброени в таблица 4.3, се записва непосредствено след B. По този начин инструкцията BEQ се използва за разклоняване при условие равно (установен Z флаг).

Инструкцията BL<cc> (Branch and Link), която запазва адреса на инструкцията непосредствено след клона в регистъра на връзките (LR или R14). Това позволява извикване на подпрограма. В точката на извикващата програма управлението може да се върне с помощта на инструкциите за разклонение и копиране на регистъра на връзките в програмата брояч (PC).

4.3.3 Скок

Всички инструкции от групите аритметични, логически и прехвърляне на данни могат да използват програмния брояч (PC или R15) като регистър на местоназначението. Това им позволява да променят хода на изпълнението, от което се извлича следващата инструкция. Директната манипулация на компютъра по този начин е известна като скок. Всяка 32-битова стойност може да бъде копирана в компютъра, като по този начин може да се извърши скок до всяко място в 4GB памет. Това е полезна функция, когато може да се изчисли местоположението на следващата инструкция.

4.4 Достъп до паметта

Инструкциите за достъп до паметта биват три групи: за достъп до регистър; регистрационен байт и множество регистри. Всяка група от тях има инструкция за зареждане и съхранение.

4.4.1 Зареждане и съхраняване на регистър

Инструкцията LDR (Load Register) може да зареди 32-битова дума от паметта в регистър. Докато инструкциите STR (Store Register) ще съхраняват 32-битова дума от регистър в паметта.

Инструкциите за зареждане и съхранение на регистъра използват стойност <op2>, за да идентифицират паметта, която да се използва. Всички <op2> стойности използват основен регистър и отместване в някаква форма, определена от инструкцията:

Отместването на адресирането е мястото, където адресът на паметта се формира чрез добавяне (или изваждане) на отместване към (или от) стойността, съхранявана в основния регистър.

Предварително индексираното адресиране е мястото, където адресът на паметта се формира по същия начин, както при адресиране с отместване. Като допълнителна екстра адресът на паметта също се записва обратно в основния регистър. Пост-индексираното адресиране указва мястото, където адресът на паметта се взема директно от основния регистър без никакви модификации. Този път основният

регистър се променя (отместването се добавя или изважда) след зареждането или запамятаването.

4.4.2 Зареждане и съхраняване на байт на регистъра

Инструкцията за зареждане и съхраняване на регистъра, инструкциите LDRB (Зареждане на байт на регистъра) и STRB (Байт за съхранение на регистър) могат да заредят 8-битов байт от паметта или съответно да запазят такъв. Местоположението на паметта се идентифицира със стойност <op2> по същия начин.

Инструкцията за зареждане на байта ще зареди байта в младшите 8 бита на регистъра. Старшите 24-бита на регистъра се нулират. Байтът не може да бъде число със знак, освен ако не се предприемат допълнителни стъпки. Когато се съхранява байт, се съхраняват само по-младшите 8 бита от регистъра. Останалите 24-бита се игнорират.

4.4.3 Зареждане и съхранение на множество регистри

Инструкциите LDM (Load Multiple) и STM (Store Multiple) позволяват блоково прехвърляне на произволен брой регистри към или от паметта. Местоположението на паметта трябва да е зададено в основен регистър, който по желание може да бъде актуализиран след прехвърлянето. Даден е списък с регистри за прехвърляне, това може да включва програмен брояч. Всеки регистър се прехвърля към или от паметта на свой ред, започвайки с най-ниския регистър (R0, ако е даден) и завършвайки с най-високия регистър (R15, ако е посочен в списъка с регистри). Базовият регистър се модифицира с прехвърляне, така че системата знае мястото на паметта за следващото прехвърляне. Базовият регистър се модифицира според един от четирите режима на трансфер:

4.5. Логически и инструкции за манипулация на битове

A	B	C	A	B	C	A	B	C	A	B	C
0	0	0	0	0	0	0	0	0	0	0	0
0	1	0	0	1	0	0	1	1	0	1	1
1	0	0	1	0	1	1	0	1	1	0	1
1	1	1	1	1	0	1	1	0	1	1	1
C = A AND B			C = A BIC B			C = A EOR B			C = A ORR B		

Table 4.4: True tables for Logical operations

IB Increment Before Увеличение преди;

IA Increment After Увеличение след;

DB Намаление преди;

DA Намаление след.

Инструкциите Load and Store Multiple са особено подходящи за запазване на регистри по време на извикване на подпрограма. Съхраняването на множество регистри позволява регистрите да бъдат записани в началото на подпрограмата, докато Load Multiple може да върне регистрите обратно в първоначалното им състояние в края на подпрограмата.

4.5 Логически и манипулация на битове

Всичките пет инструкции в тази група подготвят логическа операция върху два (или в един случай, един) изходни операнда, единият от които трябва да бъде регистър (Rn), а другият е стойност <op1>, което води до резултат, който се поставя в регистъра

на местоназначението. Те могат също така по избор да актуализират флаговете на кода на условие въз основа на резултата.

Стойност <op1> трябва да премине през Barrel Shift, преди да може да се използва. Това означава, че всяка инструкция, която използва стойност <op1>, може да извърши аритметично или логическо изместване. Това включва инструкцията MOV (Преместване). В резултат на това няма изискване за специални инструкции за смяна, присъстващи в компютрите с комплексен набор от инструкции.

Инструкцията AND (Bitwise AND) ще реализира операция и на битовите на първия операнд (регистъра) и тези на втория операнд (стойността <op1>). В резултата битът ще бъде единица само ако е единица както в първия, така и във втория операнд, в противен случай битът ще бъде нула. И операция често се използва за маскиране или изчистване на битове, които не ни интересуват.

Инструкцията BIC (Bit Clear) е предназначена да поеме функцията за маскиране от инструкцията AND. С тази инструкция в резултата се нулира бит ако битът във втория операнд е установен, без значение от стойността на бита в първия операнд. Задържащите битове се оставят непроменени.

Инструкцията EOR (Исключващо ИЛИ) ще сумира по модул 2 битовите на първия операнд с тези на втория операнд. Ако и двата бита са установени или нулирани, полученият бит на резултата ще бъде нулиран. Това е известно като Exclusive OR и може да се използва за инвертиране на избрани битове. Чрез установяване на бит във втория операнд съответният бит в първия операнд ще бъде инвертиран, ако е бил установен ще стане нула, ако е бил нула, сега ще бъде единица.

Инструкцията ORR (Bitwise OR) просто ще реализира ИЛИ операция над битовите на първия и втория операнди. Битът в резултата ще бъде единица, ако съответният бит е установен в някой от двата операнда. Ако битът е нула във втория операнд, полученият бит ще бъде същият като първия операнд. Това е известно като логическа равнозначност, тъй като установява бит, зададен не само в първия операнд или втория операнд, но и в двата операнда.

И накрая, инструкцията MVN (Move Not) е малко по-различна с това, че използва само един операнд. Резултатът е пълна инверсия на стойността на операндите. Това означава, че всеки бит се превключва от set to clear и clear to set. Това може да бъде полезно за създаване на битова маска за използване с някоя от другите инструкции.

4.6 Системен контрол / Привилегировани инструкции

4.6.1 Софтуерно прекъсване

Инструкциите SWI (Software Interrupt) причиняват изключение за софтуерно прекъсване. Те обикновено се използват за заявяване на услуга от операционна система. Изключението кара процесора да премине в привилегирован режим, като по този начин позволява на задачата в потребителския режим да получи достъп до привилегировани функции, но само под управление на операционна система. Има три метода, които операционната система използва, за да определи коя услуга се изисква.

1. Сервизен номер се предоставя като непосредствена стойност, свързана с SWI. Всички допълнителни параметри се предават между операционната система и задачата чрез регистри с общо предназначение. Това е най-често срещаната форма на комуникация.
2. Номерът на услугата се вписва в регистър с общо предназначение. Всички допълнителни параметри също се предават в регистри с общо предназначение.
3. Сервизният номер се поставя в структура от данни, която следва непосредствено след инструкцията. Допълнителни параметри могат да се предават като част от

структурата на данните или в регистри. Подробностите за SWI инструкцията зависят от операционната система или средата.

Използваната функция по-нататък има номер 0x11. В системата, която използваме, това представлява функцията `exit()` и се използва за излизане от програма.

4.6.2 Семафори

Когато има два или повече процеса, работещи на един или повече процесори, те се нуждаят от начин за комуникация, за да споделят общи ресурси. Това може да бъде предоставено от семафор. Представени са две инструкции, които биха помогнали на операционната система да предостави семафор.

Инструкцията SWP (Swap) чете стойност на дума от паметта (в регистър местоназначение), като я замества с друга стойност (от регистър източник).

Местоположението на паметта е определено в още един регистър. Един и същ регистър може да бъде даден за източник и местоназначение, като в този случай стойността в мястото на паметта и стойността на регистъра просто се обменят.

Инструкцията SWPB (Swap Byte) прави същата операция за един байт, а не за дума.

4.6.3 Достъп до регистър на състоянието

Регистърът на състоянието може да бъде достъпен чрез две инструкции.

Инструкцията MRS (Прехвърляне към регистър от състояние) ще копира текущия регистър на състоянието на процеса (CPSR) в регистър с общо предназначение. Има само една причина за четене на регистъра на състоянието и това е да промените едно от полетата. Има и съответна инструкция MSR (Преместване към състояние от регистър), която замества определени полета в текущия регистър на състоянието. Достъпът до контролните полета (режим на процес и флагове за разрешаване на прекъсване) е ограничен до привилегирован режим. Само флаговете на кода на условието могат да бъдат презаписани в потребителски режим, това е, за да попречи на своенравната програма за потребителски режим да влезе в привилегирован режим.

Инструкциите MRS и MSR могат също да имат достъп до регистъра на състоянието на запаметения процес (SPSR). Тъй като режимите на потребителя и системата нямат SPSR регистър, а привилегированите режими имат, тези инструкции са ограничени само до привилегирован режим.

Инструкциите за прехвърляне на данни, аритметичните инструкции и логическите инструкции ползват програмния брояч като регистър на местоназначението. Това им позволява да изчислят адреса на следващата инструкция. В допълнение към това, когато е даден регистърът на местоназначението и е зададен флаг <S>, тези инструкции също ще копират SPSR за текущия режим в CPSR. Очевидно такива инструкции са ограничени до привилегировани режими, тъй като те всъщност имат SPSR. Това е доста особен специален ефект, който е предоставен, за да позволи манипулатори на изключения

4.6.5 Привилегирован достъп до паметта

Инструкциите за достъп до паметта предоставят достъп до паметта в текущия режим на процесора. Когато процесорът е в привилегирован режим, достъпът до паметта също ще бъде привилегирован. Това може да причини затруднения, ако паметта, до която се осъществява достъп, е налична само в потребителски режим. За справяне с тази трудност е предоставен набор от инструкции за достъп до паметта, които осъществяват достъп до паметта в потребителски режим, без да засягат текущия режим на процесора. Те са известни като инструкциите „с прехвърляне“.

LDRT (Load Register with Translation) и STRT (Store Register with Translation) осигуряват 32-битов достъп до паметта с прехвърляне в потребителски режим. Съответните инструкции LDRBT (Зареждане на регистърен байт с прехвърляне) и STRBT (Запазване на байт на регистъра с прехвърляне) осигуряват 8-битов достъп до паметта с прехвърляне в потребителски режим. Тези инструкции се използват, когато се изисква манипулатор на изключения за достъп до паметта на потребителския режим.

Методи на адресиране

5.1 Операнди за обработка на данни: <op>

По-голямата част от инструкциите се отнасят до обработка на данни под някаква форма. Един от операндите към тези инструкции се пренасочва през Barrel Shifter. Това означава, че операндът може да бъде модифициран, преди да бъде използван. Това може да бъде много полезно при работа със списъци, таблици и други сложни структури от данни. Обозначаваме инструкции от този тип като вземащи един от аргументите си от <op1>. Аргументът <op1> може да идва от един от двата източника, константна стойност или регистър, и да бъде модифициран по пет различни начина.

5.1.1 Непроменена стойност

Можете да използвате непроменена стойност или регистър, като просто дадете стойността или името на регистъра. Например следните инструкции демонстрират двата метода:

Непосредствено

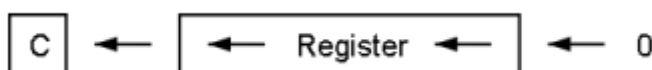
MOV R0, #1234 ;R0 <-IR(стойност)

Ще премести непосредствената константна стойност 123410 в регистъра R0.

Регистрово

MOV R0, R1 ;R0 <- R1

Ще премести стойността в регистъра R1 в регистъра R0.



Това ще приеме стойността на регистър и ще измести стойността нагоре, към най-значимия бит, с n бита. Броят на битовете за изместване се определя или от константна стойност, или от друг регистър. Долните битове на стойността се заменят с нула. Това е прост начин за извършване на умножение по степен на 2 ($\times 2^n$).

Логическо изместване наляво непосредствено

MOV R0, R1, LSL #2 $R0 \leftarrow R1 \ll IR(\text{value})$

R0 ще получи стойността на R1, изместена наляво на 32 бита. Стойността на R1 не се поменя.

Логическо изместване наляво регистрово

MOV R0, R1, LSL R2 $R0 \leftarrow R1 \ll R2(7:0)$

R0 ще стане стойността на R1, изместена наляво с броя битове, посочени в регистъра R2. R0 е единственият регистър за промяна, както R1, така и R2 не се променят от тази операция. Ако инструкцията е да зададете регистъра на състоянието, флагът за пренасяне (C) е последният бит, който е изместен извън стойността.

5.1.3 Логическо изместване надясно



Логическото отместване надясно е много подобно на логическото отместване наляво, с изключение на това, че ще измести стойността надясно, към най-малко значимия бит, с n бита. Той ще замени горните битове с нули, като по този начин ще осигури ефективно деление без знак на 2^n функция ($| \div 2^n |$). Броят на битовете за изместване може да бъде определен чрез константна стойност или друг регистър.

Логическо изместване надясно непосредствено

MOV R0, R1, LSR #2 $R0 \leftarrow R1 \gg IR(\text{value})$

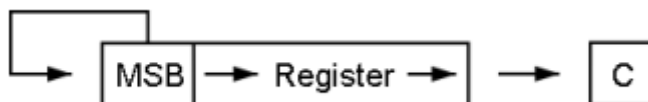
R0 ще приеме стойността на R1, изместена надясно с 2 бита. Стойността на R1 не се променя

Логическо изместване надясно регистрово

MOV R0, R1, LSR R2 $R0 \leftarrow R1 \gg R2(7:0)$

Както преди R0 ще стане стойността на R1, изместена надясно с броя битове, посочени в регистъра R2. R1 и R2 не се променят от тази операция. Ако инструкцията е да зададете регистъра на състоянието, флагът за пренос (C) е последният бит, който трябва да бъде изместен извън стойността.

5.1.4 Аритметично изместване надясно



Аритметичното изместване вдясно е доста подобно на логическото отместване вдясно, но вместо да замества горните битове с нула, той поддържа стойността на най-значимия бит. Тъй като най-старшият бит се използва за запис на знака, това означава, че знакът на стойността се поддържа, като по този начин се осигурява деление със знак на 2^n операция ($\div 2^n$).

Аритметично изместване надясно непосредствено

MOV R0, R1, ASR #2 $R0 \leftarrow R1 \ggg IR(\text{value})$

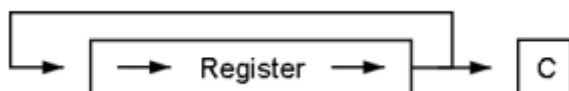
В регистър R0 се помещава стойността на регистър R1, изместена надясно с 2 бита, със запазване на знака.

Аритметично изместване надясно регистрово

MOV R0, R1, ASR R2 $R0 \leftarrow R1 \ggg R2$

В регистър R0 се записва стойността на регистъра R1, изместен надясно с броя битове, посочени от регистъра R2. R1 и R2 не се променят от тази операция. Има инструкции логическо и аритметично изместване надясно, но няма аритметично изместване наляво. Знакът на числото, записано в допълнителен код, се съхранява в най-старшите битове на резултата. Няма аритметично изместване наляво, тъй като знакът се запазва автоматично от логическото отместване.

5.1.5 Ротация надясно



При операцията ротация надясно най-младшият бит се копира във флага за пренос (C), докато стойността на флага C се копира в най-старшия бит от числото. По този начин нито един от битовете в числото не се сменя, а просто се премества от младшите битове към старшите битове на числото.

Ротация надясно непосредствена

MOV R0, R1, ROR #2 $R0 \leftarrow R1 \ggg IR(value)$

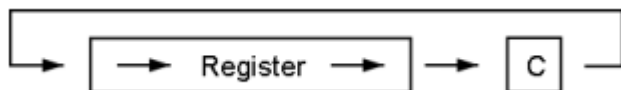
Това ще завърти стойността на R1 с два бита. Най-старшият бит от получената стойност ще бъде същият като най-младшия бит от първоначалната стойност. Вторият бит след най-старшия ще бъде същият като флага Carry. Във формат S флагът Carry се записва във втория най-младши бит от оригиналната стойност. Стойността на R1 не се променя при тази операция.

Ротация надясно регистрова

MOV R0, R1, ROR R2 $R0 \leftarrow R1 \ggg R2$

В регистър R0 се записва стойността на регистъра R1, завъртяна надясно с броя битове, посочени от регистъра R2. R1 и R2 не се променят от тази операция. Няма инструкция завъртане наляво. Събирането с пренос (ADC) с нула предоставя тази услуга за един бит. В набора от инструкции няма инструкция ROL.

5.1.6 Изместване надясно разширено



Това е подобно на завъртане надясно с един бит. Името разширено идва от факта, че тази функция премества стойността на флага за пренос (C) в най-старшия бит от стойността и най-младшия бит от стойността отива във флага за пренос (C). По този начин флагът Carry може да се копира през числа, записани в няколко думи. Така в изчисленията могат да се използват числа, по-големи от 32 бита.

Ротация надясно разширена

MOV R0, R1, RRX $R0 \leftarrow C \ggg R1 \ggg C$

Регистърът R0 получава стойността на регистъра R1, завъртяна през флага за пренос на един бит. Най-старшият бит от стойността става същият като текущия флаг за пренос, докато флагът за пренос ще бъде същият като най-младшия бит на R1. Стойността на R1 не се променя.

5.2 Операнди при достъп до паметта: <op2>

Адресът на паметта, използван в инструкциите за достъп до паметта, може също да бъде модифициран. Това осигурява по-гъвкав достъп до паметта, което е особено полезно при работа с по-сложни структури от данни. Той позволява инструкции с пред- и след- инкрементация да актуализират указателите на паметта като страничен ефект от инструкцията. Това прави циклите за достъп до паметта, по-ефективни.

При инструкции от този тип един от аргументите се извлича от <op2>. Има три основни метода за определяне на адрес на паметта (<op2>), всички от които включват стойност на отнемстване под някаква форма. Това изместване може да бъде определено по един от три начина:

Константа

Постоянна стойност. Ако не е посочено изместване, се приема непосредствена постоянна стойност нула.

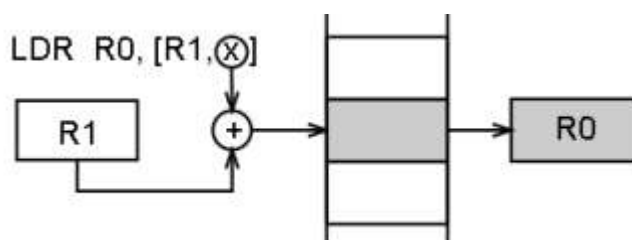
Регистър

Отместването може да бъде извлечено от регистър. Стойността на регистъра се добавя към адреса, съхраняван в друг регистър, за да се получи крайния адрес.

Машаб

Отместването се получава от регистър, който може да бъде мащабиран от един от операторите на смяна, използвани за <op1>. По-конкретно от операторите за смяна на логическо изместване наляво (LSL), логическо изместване надясно (LSR), аритметично изместване надясно (ASR), ротация надясно (ROR) или разширена ротация надясно (RRX), където броят на битовете за изместване е посочен като константа.

5.2.1 Адресиране с отместване



При адресирането с отместване адресът на паметта се формира чрез добавяне (или изваждане) на отместване към/от стойността, съхранена в базов регистър.

Нулево отместване

LDR R0, [R1]

Ще зареди регистъра R0 с 32-битова дума на адреса на паметта, съхраняван в регистъра R1. В тази инструкция няма посочено отместване, така че се приема отместване нула. Стойността на R1 не се променя.

Непосредствено отместване

LDR R0, [R1, #4]

Ще зареди регистъра R0 с думата в адреса на паметта, изчислен чрез добавяне на константна стойност 4 към адреса на паметта, съдържащ се в регистъра R1. Регистърът R1 не се променя от тази инструкция.

Регистрово отместване

LDR R0, [R1, R2]

Зарежда регистъра R0 със стойността в адреса на паметта, изчислена чрез добавяне на стойността в регистъра R1 към стойността, съхранявана в регистъра R2. И R1, и R2 не се променят от тази операция.

Отместване в мащабиран регистър

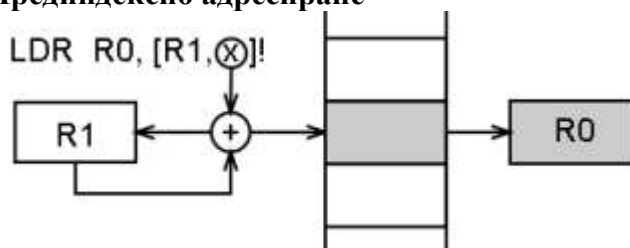
LDR R0, [R1, R2, LSL #2]

Ще зареди регистъра R0 с 32-битовата стойност на адреса на паметта, изчислен чрез добавяне на стойността в регистъра R1 към стойността, получена чрез изместване на стойността в R2 наляво с 2 бита. И двата регистъра, R1 и R2, не се променят от тази операция. Това е полезно за индексване в сложна структура от данни. Началото на структурата от данни се съхранява в основен регистър, R1 в този случай, а отместването за достъп до конкретно поле в структурата след това се добавя към

основния адрес. Зареждането на отместването в регистър дава възможност то да бъде изчислено по време на изпълнение, а не да е фиксирано.

Мащабирана стойност може да се използва и за достъп до конкретен елемент от таблица, където размерът на елемента е степен на две. Например за да намерим елемент 7 в таблица с 32-битови стойности, трябва само да изместим стойността на индекса 6 наляво с 2 бита (6×2^2), за да изчислим стойността, която трябва да добавим като отместване към началото на таблицата в регистъра R1 в нашия пример. Компютърът отброява от нула, така че използваме индексна стойност от 6, а не 7. 32-битовото число изисква 4 байта памет, което е 2^2 , така че се нуждаем само от 2-битово изместване наляво.

5.2.2 Прединдексно адресиране



При предварителното индексно адресиране адресът на паметта се получава по същия начин, както при адресиране с отместване. Адресът се използва не само за достъп до паметта, но и основният регистър се променя, за да съхрани новата стойност. В ARM системата това е известно като обратно запис и се обозначава чрез поставяне на удивителен знак след в края на кода <op2>. Прединдексното адресиране може да бъде полезно в цикъл, тъй като може да се използва за автоматично увеличаване или намаляване на брояч или указателя на паметта.

Непосредствено прединдексно адресиране

LDR R0, [R1, #4]!

Ще зареди регистъра R0 с думата на адреса на паметта, изчислен чрез добавяне на константна стойност 4 към адреса на паметта, съдържащ се в регистъра R1. Новият адрес на паметта се записва обратно в основния регистър, регистър R1.

Регистрово прединдексно адресиране

LDR R0, [R1, R2]!

При предварително индексно адресиране адресът на паметта, се формира по същия начин, както при адресирането с отместване. Адресът се използва за достъп до паметта, но и основният регистър се променя, за да съхрани новата стойност.

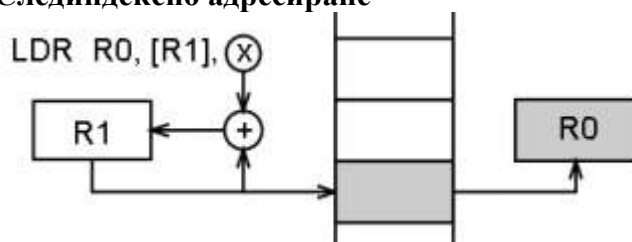
В ARM системата това е известно като обратно запис и се обозначава чрез поставяне на удивителен знак след края на кода <op2>. Предварителният индекс може да бъде особено полезен в цикъл, тъй като може да се използва за автоматично увеличаване или намаляване на брояч или указател на паметта.

Мащабирано регистрово прединдексно адресиране

LDR R0, [R1, R2, LSL #2]!

Първо изчислява новия адрес чрез добавяне на стойността в основния адресен регистър R1, към стойността, получена чрез изместване на стойността в отместващия регистър, R2, наляво с 2 бита. След това зарежда получения 32-битов адрес в регистъра на местоназначението, R0. Новият адрес също се записва обратно в основния регистър, R1. Стойността на регистъра за отместване, R2, няма да бъде променена от тази операция.

5.2.3 Следииндексно адресиране



При този начин на адресиране адресът на паметта е стойността на основния регистър. Като страничен ефект, отместването се добавя към или изважда от стойността на основния регистър и резултатът се записва обратно в основния регистър. Следииндексното адресиране използва стойността на основния регистър без промяна. След това прилага модификацията на адреса и новия адрес се записва обратно в основния регистър. Това може да се използва за автоматично увеличаване или намаляване на указател на паметта, след като е бил използван, така че да сочи към следващото място, което да се адресира. Тъй като инструкцията подготвя обратен запис, не се включва удивителен знак, а само се премества затварящата скоба, за да се включи само основния регистър, който е регистърът, съдържащ адреса на паметта, към който се обръщаме.

Непосредствено следииндексно адресиране

LDR R0, [R1], #4

Ще зареди регистъра R0 с думата на адреса на паметта, съдържащ се в основния регистър R1. След това ще изчисли новата стойност на R1, като добави константа 4 към текущата стойност на R1.

Регистрово следииндексно адресиране

LDR R0, [R1], R2

Зарежда регистъра R0 със стойността на адреса на паметта, съхраняван в основния регистър R1. След това изчислява новата стойност на основния регистър, като добавя стойността в отместващия регистър R2, към текущата стойност в основния регистър. Регистърът за отместване R2, не се променя от тази операция.

Масшабирано регистрово следииндексно адресиране

LDR R0, [R1], R2, LSL #2

Първо зарежда 32-битовата стойност на адреса на паметта, съдържащ се в основния регистър R1, в регистъра на получателя R0. След това изчислява новата стойност за основния регистър, като добавя текущата стойност към стойността, получена чрез изместване на стойността в отместващия регистър R2, наляво на 2 бита. Регистърът за отместване R2, няма да се промени от тази операция.